

which is useful in proving the recurrences

$$\begin{aligned}
 D_{m+1,i} &= \frac{C_{m,i+1}(C_{m,i+1} - D_{m,i})}{\left(\frac{x-x_i}{x-x_{i+m+1}}\right) D_{m,i} - C_{m,i+1}} \\
 C_{m+1,i} &= \frac{\left(\frac{x-x_i}{x-x_{i+m+1}}\right) D_{m,i} (C_{m,i+1} - D_{m,i})}{\left(\frac{x-x_i}{x-x_{i+m+1}}\right) D_{m,i} - C_{m,i+1}}
 \end{aligned} \tag{3.4.8}$$

The class for rational function interpolation is identical to that for polynomial interpolation in every way, except, of course, for the different method implemented in `rawinterp`. See the end of §3.2 for usage. Plausible values for M are in the range 4 to 7.

`interp_1d.h`

`struct Rat_interp : Base_interp`

Diagonal rational function interpolation object. Construct with **x** and **y** vectors, and the number **m** of points to be used locally, then call `interp` for interpolated values.

```
{
    Doub dy;
    Rat_interp(VecDoub_I &xv, VecDoub_I &yv, Int m)
        : Base_interp(xv,&yv[0],m), dy(0.) {}
    Doub rawinterp(Int j1, Doub x);
};
```

`Doub Rat_interp::rawinterp(Int j1, Doub x)`

Given a value **x**, and using pointers to data **xx** and **yy**, this routine returns an interpolated value **y**, and stores an error estimate **dy**. The returned value is obtained by **mm**-point diagonal rational function interpolation on the subrange `xx[j1..j1+mm-1]`.

```
{
    const Doub TINY=1.0e-99;          A small number.
    Int m,i,ns=0;
    Doub y,w,t,hh,h,dd;
    const Doub *xa = &xx[j1], *ya = &yy[j1];
    VecDoub c(mm),d(mm);
    hh=abs(x-xa[0]);
    for (i=0;i<mm;i++) {
        h=abs(x-xa[i]);
        if (h == 0.0) {
            dy=0.0;
            return ya[i];
        } else if (h < hh) {
            ns=i;
            hh=h;
        }
        c[i]=ya[i];
        d[i]=ya[i]+TINY;
    }
    y=ya[ns--];
    for (m=1;m<mm;m++) {
        for (i=0;i<mm-m;i++) {
            w=c[i+1]-d[i];
            h=xa[i+m]-x;
            t=(xa[i]-x)*d[i]/h;
            dd=t-c[i+1];
            if (dd == 0.0) throw("Error in routine ratint");
            This error condition indicates that the interpolating function has a pole at the requested value of x.
            dd=w/dd;
            d[i]=c[i+1]*dd;
            c[i]=t*dd;
```

The TINY part is needed to prevent a rare zero-over-zero condition.

h will never be zero, since this was tested in the initializing loop.

```

    }
    y += (dy=(2*(ns+1) < (mm-m) ? c[ns+1] : d[ns--]));
}
return y;
}

```

3.4.1 Barycentric Rational Interpolation

Suppose one tries to use the above algorithm to construct a global approximation on the entire table of values using all the given nodes $x_0, x_1, \dots, x_{N-1}$. One potential drawback is that the approximation can have poles inside the interpolation interval where the denominator in (3.4.1) vanishes, even if the original function has no poles there. An even greater (related) hazard is that we have allowed the order of the approximation to grow to $N - 1$, probably much too large.

An alternative algorithm can be derived [2] that has no poles anywhere on the real axis, and that allows the actual order of the approximation to be specified to be any integer $d < N$. The trick is to make the degree of both the numerator and the denominator in equation (3.4.1) be $N - 1$. This requires that the p 's and the q 's not be independent, so that equation (3.4.2) no longer holds.

The algorithm utilizes the *barycentric form* of the rational interpolant

$$R(x) = \frac{\sum_{i=0}^{N-1} \frac{w_i}{x - x_i} y_i}{\sum_{i=0}^{N-1} \frac{w_i}{x - x_i}} \quad (3.4.9)$$

One can show that by a suitable choice of the weights w_i , every rational interpolant can be written in this form, and that, as a special case, so can polynomial interpolants [3]. It turns out that this form has many nice numerical properties. Barycentric rational interpolation competes very favorably with splines: its error is often smaller, and the resulting approximation is infinitely smooth (unlike splines).

Suppose we want our rational interpolant to have approximation order d , i.e., if the spacing of the points is $O(h)$, the error is $O(h^{d+1})$ as $h \rightarrow 0$. Then the formula for the weights is

$$w_k = \sum_{\substack{i=k-d \\ 0 \leq i < N-d}}^k (-1)^k \prod_{\substack{j=i \\ j \neq k}}^{i+d} \frac{1}{x_k - x_j} \quad (3.4.10)$$

For example,

$$\begin{aligned} w_k &= (-1)^k, & d &= 0 \\ w_k &= (-1)^{k-1} \left[\frac{1}{x_k - x_{k-1}} + \frac{1}{x_{k+1} - x_k} \right], & d &= 1 \end{aligned} \quad (3.4.11)$$

In the last equation, you omit the terms in w_0 and w_{N-1} that refer to out-of-range values of x_k .

Here is a routine that implements barycentric rational interpolation. Given a set of N nodes and a desired order d , with $d < N$, it first computes the weights w_k . Then subsequent calls to `interp` evaluate the interpolant using equation (3.4.9). Note that the parameter `j1` of `rawinterp` is not used, since the algorithm is designed to construct an approximation on the entire interval at once.

The workload to construct the weights is of order $O(Nd)$ operations. For small d , this is not too different from splines. Note, however, that the workload for each subsequent interpolated value is $O(N)$, not $O(d)$ as for splines.

interp_1d.h

```
struct BaryRat_interp : Base_interp
```

Barycentric rational interpolation object. After constructing the object, call `interp` for interpolated values. Note that no error estimate `dy` is calculated.

```
{
    VecDoub w;
    Int d;
    BaryRat_interp(VecDoub_I &xv, VecDoub_I &yv, Int dd);
    Doub rawinterp(Int j1, Doub x);
    Doub interp(Doub x);
};
```

```
BaryRat_interp::BaryRat_interp(VecDoub_I &xv, VecDoub_I &yv, Int dd)
    : Base_interp(xv,&yv[0],xv.size()), w(n), d(dd)
```

Constructor arguments are **x** and **y** vectors of length n , and order d of desired approximation.

```
{
    if (n<=d) throw("d too large for number of points in BaryRat_interp");
    for (Int k=0;k<n;k++) {
        Int imin=MAX(k-d,0);
        Int imax = k >= n-d ? n-d-1 : k;
        Doub temp = imin & 1 ? -1.0 : 1.0;
        Doub sum=0.0;
        for (Int i=imin;i<=imax;i++) {
            Int jmax=MIN(i+d,n-1);
            Doub term=1.0;
            for (Int j=i;j<=jmax;j++) {
                if (j==k) continue;
                term *= (xx[k]-xx[j]);
            }
            term=temp/term;
            temp=-temp;
            sum += term;
        }
        w[k]=sum;
    }
}
```

```
Doub BaryRat_interp::rawinterp(Int j1, Doub x)
```

Use equation (3.4.9) to compute the barycentric rational interpolant. Note that `j1` is not used since the approximation is global; it is included only for compatibility with `Base_interp`.

```
{
    Doub num=0,den=0;
    for (Int i=0;i<n;i++) {
        Doub h=x-xx[i];
        if (h == 0.0) {
            return yy[i];
        } else {
            Doub temp=w[i]/h;
            num += temp*yy[i];
            den += temp;
        }
    }
    return num/den;
}

Doub BaryRat_interp::interp(Doub x) {
    No need to invoke hunt or locate since the interpolation is global, so override interp to simply
    call rawinterp directly with a dummy value of j1.
    return rawinterp(1,x);
}
```

It is wise to start with small values of d before trying larger values.

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §2.2.[1]
- Floater, M.S., and Hormann, K. 2006+, “Barycentric Rational Interpolation with No Poles and High Rates of Approximation,” at <http://www.in.tu-clausthal.de/fileadmin/homes/techreports/ifi0606hormann.pdf>. [2]
- Berrut, J.-P., and Trefethen, L.N. 2004, “Barycentric Lagrange Interpolation,” *SIAM Review*, vol. 46, pp. 501–517. [3]
- Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), §6.2.
- Cuyt, A., and Wuytack, L. 1987, *Nonlinear Methods in Numerical Analysis* (Amsterdam: North-Holland), Chapter 3.

3.5 Coefficients of the Interpolating Polynomial

Occasionally you may wish to know not the value of the interpolating polynomial that passes through a (small!) number of points, but the coefficients of that polynomial. A valid use of the coefficients might be, for example, to compute simultaneous interpolated values of the function and of several of its derivatives (see §5.1), or to convolve a segment of the tabulated function with some other function, where the moments of that other function (i.e., its convolution with powers of x) are known analytically.

Please be certain, however, that the coefficients are what you need. Generally the coefficients of the interpolating polynomial can be determined much less accurately than its value at a desired abscissa. Therefore, it is not a good idea to determine the coefficients only for use in calculating interpolating values. Values thus calculated will not pass exactly through the tabulated points, for example, while values computed by the routines in §3.1 – §3.3 will pass exactly through such points.

Also, you should not mistake the interpolating polynomial (and its coefficients) for its cousin, the *best-fit* polynomial through a data set. Fitting is a *smoothing* process, since the number of fitted coefficients is typically much less than the number of data points. Therefore, fitted coefficients can be accurately and stably determined even in the presence of statistical errors in the tabulated values. (See §14.9.) Interpolation, where the number of coefficients and number of tabulated points are equal, takes the tabulated values as perfect. If they in fact contain statistical errors, these can be magnified into oscillations of the interpolating polynomial in between the tabulated points.

As before, we take the tabulated points to be $y_i \equiv y(x_i)$. If the interpolating polynomial is written as

$$y = c_0 + c_1x + c_2x^2 + \cdots + c_{N-1}x^{N-1} \quad (3.5.1)$$

then the c_i 's are required to satisfy the linear equation

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{N-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{N-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{N-1} & x_{N-1}^2 & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix} \quad (3.5.2)$$

This is a *Vandermonde matrix*, as described in §2.8. One could in principle solve equation (3.5.2) by standard techniques for linear equations generally (§2.3); however, the special method that was derived in §2.8 is more efficient by a large factor, of order N , so it is much better.

Remember that Vandermonde systems can be quite ill-conditioned. In such a case, *no* numerical method is going to give a very accurate answer. Such cases do not, please note, imply any difficulty in finding interpolated *values* by the methods of §3.2, but only difficulty in finding *coefficients*.

Like the routine in §2.8, the following is due to G.B. Rybicki.

polcoef.h

```
void polcoe(VecDoub_I &x, VecDoub_I &y, VecDoub_0 &cof)
Given arrays x[0..n-1] and y[0..n-1] containing a tabulated function  $y_i = f(x_i)$ , this routine
returns an array of coefficients cof[0..n-1], such that  $y_i = \sum_{j=0}^{n-1} \text{cof}_j x_i^j$ .
{
    Int k,j,i,n=x.size();
    Doub phi,ff,b;
    VecDoub s(n);
    for (i=0;i<n;i++) s[i]=cof[i]=0.0;
    s[n-1] = -x[0];
    for (i=1;i<n;i++) {
        for (j=n-1-i;j<n-1;j++)
            s[j] -= x[i]*s[j+1];
        s[n-1] -= x[i];
    }
    for (j=0;j<n;j++) {
        phi=n;
        for (k=n-1;k>0;k--)
            phi=k*s[k]+x[j]*phi;
        ff=y[j]/phi;
        b=1.0;
        for (k=n-1;k>=0;k--) {
            cof[k] += b*ff;
            b=s[k]+x[j]*b;
        }
    }
}
```

Coefficients s_i of the master polynomial $P(x)$ are found by recurrence.

The quantity $\text{phi} = \prod_{j \neq k} (x_j - x_k)$ is found as a derivative of $P(x_j)$.

Coefficients of polynomials in each term of the Lagrange formula are found by synthetic division of $P(x)$ by $(x - x_j)$. The solution c_k is accumulated.

3.5.1 Another Method

Another technique is to make use of the function value interpolation routine already given (polint; §3.2). If we interpolate (or extrapolate) to find the value of the interpolating polynomial at $x = 0$, then this value will evidently be c_0 . Now we can subtract c_0 from the y_i 's and divide each by its corresponding x_i . Throwing out one point (the one with smallest x_i is a good candidate), we can repeat the procedure to find c_1 , and so on.

It is not instantly obvious that this procedure is stable, but we have generally found it to be somewhat *more* stable than the routine immediately preceding. This method is of order N^3 , while the preceding one was of order N^2 . You will find, however, that neither works very well for large N , because of the intrinsic ill-condition of the Vandermonde problem. In single precision, N up to 8 or 10 is satisfactory; about double this in double precision.